OpenEBFM: Energy-based Foundation Models Are Scalable Architectures

Yuan Pu*§, Yudong Lu*, Ting Wang*, Zhicong Hong§, Qinuo Li, Xuansheng Wu, Na Zou, Xia Hu, Guanchu Wang

Shanghai Artificial Intelligence Laboratory

{puyuan,luyudong,wangting1,hongzhicong,liqinuo, wuxuansheng,zouna,huxia,wangguanchu}@pjlab.org.cn

https://github.com/datamllab/openebfm

https://huggingface.co/collections/guan-wang/openebfm

Abstract

Energy-based models have shown promising properties for domain-specific tasks but remain challenging in modeling general-purpose scenarios. To bridge this gap, we design Energy-Based Foundation Models (EBFMs) for general language modeling. EBFMs integrate energy steering and token generation within a single architecture to leverage the energy-guided search to refine token generation. Specifically, EBFMs first optimize a latent state over the energy landscape, then leverages the refined latent state for token generation. By performing such energy-guided pre-training on large-scale corpora, EBFMs learn generally effective energy landscape where favorable tokens correspond to lower energy values. To explore the scaling behavior of EBFMs, we pre-train EBFMs on three large-scale corpora, then evaluate across 26 downstream tasks spanning a broad range of domains. Our results demonstrate that EBFMs have promising scaling behaviors with increasing model size and training data, and outperform GPTs on most downstream tasks. These findings highlight EBFMs as a promising architecture for large-scale pre-training and general purpose problem solving. We fully release the model weights and training infrastructure to enable the community to completely reproduce and extend our results.

1 Introduction

Energy-based models (EBMs) show potential in modeling complex dependencies and constraints in problem-solving [1; 2; 3]. Instead of directly parameterizing a normalized probability distribution, EBMs assign energy values to each possible solution, and learn an energy landscape such that desirable solutions receive low energy and undesirable ones receive high energy [4; 5; 6; 7]. This formulation has shown promise in several domains, including pattern recognition [8; 9], structured prediction [10], robot controlling [11; 12], and sequential planning [13]. These applications indicate that energy-based modeling offers an effective alternative paradigm for learning and inference [14].

However, existing EBMs are typically designed and evaluated within a single task or domain, such as representation learning over specific data distributions [7], structured understanding and prediction for particular tasks [15; 16], or control and planning in specialized environments [12; 11]. As a result, EBMs remain specialized to particular domains, while their generalizability across diverse tasks and domains remains largely unexplored. In contrast, advanced foundation models, including generative language models (GPT) [17] and diffusion language models (DLM) [18], have demonstrated strong generalizability across heterogeneous tasks and contexts [17; 19; 18; 20]. Whether EBMs can achieve such general-purpose capabilities has yet to be established. Therefore, it remains an open and fundamental question: ***Can EBMs serve as foundation models for general-purpose tasks?***

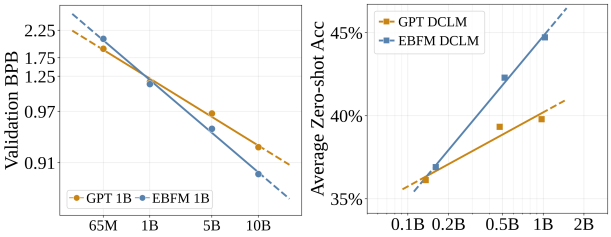
Scaling EBMs to general-purpose tasks is non-trivial problem. A fundamental challenge is that traditional EBMs are designed to estimate scalar energy values rather than directly generating structured outputs [1; 5; 21]. Therefore, it remains challenging to leverage the energy evaluation capability of EBMs to improve token generation in foundation

*Equal contribution to this work.

§Work completed while at Shanghai Artificial Intelligence Laboratory.

models [3; 22]. To address this challenge, we design the Energy-based Foundation Models (EBFMs), a novel architecture that integrates energy steering with token generation to enable general-purpose capability in EBMs. Specifically, EBFMs have two complementary components: an energy head and a generative head. During inference, the energy head estimates the energy of the input context and iteratively steers a latent variable toward lower-energy regions. After this energy-steering step, the generative head takes the steered variable and context embedding to predict new tokens. In this way, EBFMs significantly bridge the gap between energy evaluation and generative modeling, enabling the model to acquire broad capabilities through large-scale pre-training. Moreover, by refining the latent representations toward lower-energy states, EBFMs can better capture contextual dependencies and consistency in large-scale corpora, improving their generalization to diverse general-purpose tasks.

Beyond designing the architecture of EBFM, we systematically evaluate its scaling relation between the numbers of its parameters and training samples and its downstream task performs to demonstrate its efficiency on using training data against other architectures. Specifically, we pre-train EBFMs on three large-scale corpora [23; 24; 25], and the results demonstrate promising scaling behavior with increasing model size and training data, as shown in Figure 1. In addition, we evaluate EBFMs across 26 downstream tasks spanning a broad range of domains under zero-shot settings, and exhibit consistent advantages over GPTs. These findings establish EBFMs as a promising architecture for large-scale pre-training and general-purpose problem solving. Finally, we fully release the training recipe, infrastructure, and other details, enabling the community to completely reproduce and extend our results.



(a) Training tokens (b) Model size

Figure 1: Scaling behavior of EBFMs on the DCLM dataset.

To summarize, the contributions of this work are as follows:

- **Energy-based Foundation Models.** We design EBFMs, an energy-based architecture for foundation language modeling. EBFMs leverage energy-steering to refine token generation, capturing contextual dependencies more effectively.
- **Scaling Law.** EBFMs exhibit favorable scaling behavior with increasing model size and training data, compared with Generative Pre-trained Transformers (GPTs), and Masked Diffusion Language Models (MDLMs).
- **General-Purpose Task Solving.** EBFMs exhibit promising zero-shot performance across 26 downstream tasks spanning a broad range of domains, as measured by validation error and accuracy.
- **Open-source Ecosystem.** We fully release the model weights (OpenEBFM), training recipe, and infrastructure, such that the community can completely reproduce and extend this work.

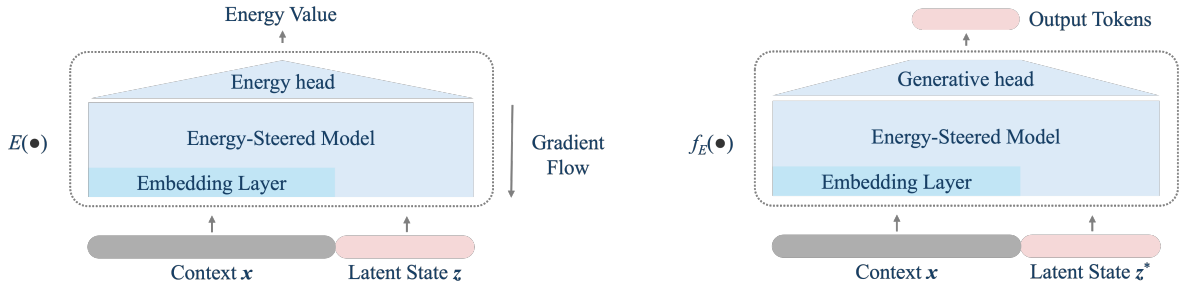
2 Preliminary

2.1 Notations

We specify the notation in this work. The overall framework of EBFM is given in Figure 2. Specifically, EBFM takes two inputs: a context sequence $\boldsymbol{x} \in \mathcal{X}$ and a latent variable $\boldsymbol{z} \in \mathcal{R}^D$, where $\mathcal{X}$ denotes the sequence space and D denotes the embedding dimension. Given these inputs, EBFM performs both energy evaluation and token generation based on an energy head and a generative head, respectively. As in Figure 2, the energy head is a linear layer for energy evaluation, while the generative head is an embedding layer with `softmax` operations for token generation. Both heads share a unified Transformer backbone, which enables energy evaluation and token generation within a single architecture. Let $E(\boldsymbol{x}, \boldsymbol{z} \mid \theta) \in \mathbb{R}$ denote the forward pass through the energy head, which outputs an energy score, as shown in Figure 2 (a). Let $f_E(\boldsymbol{x}, \boldsymbol{z} \mid \theta) \in \mathbb{V}$ denote the forward pass through the generative head, which outputs tokens from the vocabulary space $\mathbb{V}$, as illustrated in Figure 2 (b). We develop pre-training processes such that energy evaluation helps refine token generation across general-purpose scenarios.

2.2 Energy-based Models

Energy-based models (EBMs) focus on learning an energy landscape over the solution space, assigning low energy to desirable solutions and high energy to undesirable ones. However, applying EBFMs as foundation models to general-purpose tasks remains challenging, because they naturally serve as critics that evaluate candidate solutions



(a) Energy steering (b) Token generation

Figure 2: Generation process of EBFMs. (a) **Energy steering:** The energy head estimates the energy of the latent state and backpropagates the energy gradient with respect to $\boldsymbol{z}$ to obtain a low-energy latent state $\boldsymbol{z}^*$ for token generation. (b) **Token generation:** The generative head uses the optimized latent state $\boldsymbol{z}^*$ to generate the next token.

rather than directly generating outputs [1; 5; 21]. Bridging this gap is non-trivial, as fully enumerating the token space to identify the lowest-energy solution is computationally infeasible. Our proposed EBFMs address this challenge by leveraging energy evaluation to refine token generation, thus enabling general-purpose task solving.

3 Energy-based Foundation Models

We design the architecture of EBFMs in this section, including EBFM inference and training approaches.

3.1 EBFM Inference

As shown in Figure 2, EBFM inference consists of two key steps: ***energy steering*** and ***token generation***. Energy steering captures the contextual dependencies to refine token generation.

Energy Steering. EBFMs perform energy steering to refine a latent state $\boldsymbol{z}$ to maximize contextual consistency. Intuitively, $\boldsymbol{z}$ serves as a controllable representation that influences the subsequent tokens. Given $\boldsymbol{x}$ and $\boldsymbol{z}$, the energy head $E(\boldsymbol{x}, \boldsymbol{z} \mid \theta)$ evaluates the compatibility between $\boldsymbol{x}$ and $\boldsymbol{z}$, where lower energy indicates a better latent state under the current context. Energy steering aims to identify a low-energy latent state $\boldsymbol{z}^*$ that effectively captures the contextual dependencies required for subsequent token generation. Formally, the energy steering objective is given as follows:

$$\boldsymbol{z}^* = \arg\min_{\boldsymbol{z} \in \mathcal{R}^D} E(\boldsymbol{x}, \boldsymbol{z} \mid \theta) \tag{1}$$

Note that the exact solution of Equation (1) is intractable, EBFM approximates it through a K-step Markov chain Monte Carlo (MCMC) sampling. Specifically, MCMC sampling starts from an initial latent variable sampled from a standard Gaussian distribution $\boldsymbol{z}^0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, then update the latent variable through gradient descent as follows:

$$\boldsymbol{z}^k = \boldsymbol{z}^{k-1} - \alpha \nabla_{\boldsymbol{z}^{k-1}} E(\boldsymbol{x}, \boldsymbol{z}^{k-1} \mid \theta), \tag{2}$$

where α controls the step size. We omit the stochastic noise commonly used in the MCMC sampling, as we empirically observe that deterministic updates yield more stable optimization in large-scale pre-training. During energy steering, the model parameters θ remain fixed, and gradients are computed only with respect to the latent state $\boldsymbol{z}$. After K step refinement, the optimized latent state is approximated by the final state, i.e., $\boldsymbol{z}^* \approx \boldsymbol{z}^K$.

Token Generation. EBFM leverages the low-energy latent state $\boldsymbol{z}^*$ to generate subsequent tokens. Specifically, given the context $\boldsymbol{x}$, the generative head $f_E(\bullet \mid \theta)$ produce new tokens as follows:

$$\hat{\boldsymbol{y}} \sim f_E(\boldsymbol{x}, \boldsymbol{z}^* \mid \theta), \tag{3}$$

where $f_E(\boldsymbol{x}, \boldsymbol{z}^* \mid \theta)$ produces the distribution of new tokens over the entire vocabulary space. We only consider text data in this work, where $\boldsymbol{x}$ and $\boldsymbol{y}$ are text tokens.

3.2 EBFM Training

EBFMs parallelize energy steering and token generation during training to improve training efficiency.

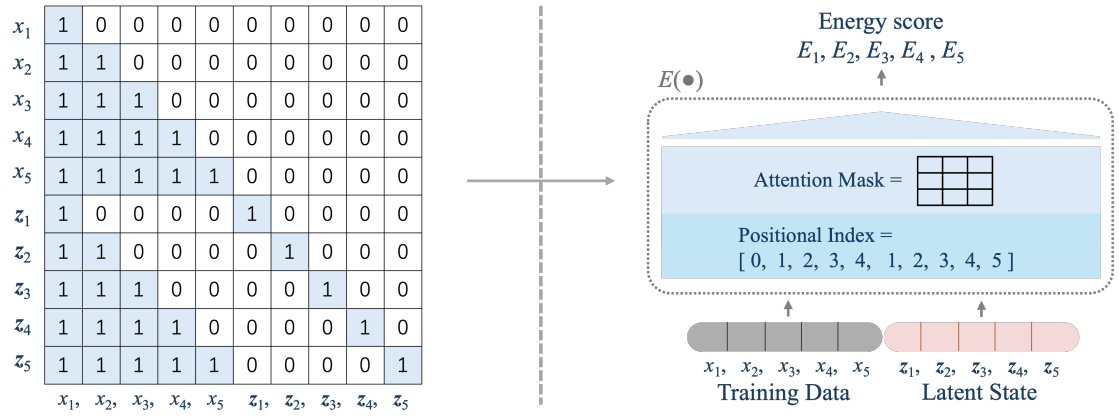


Figure 3: EBFMs parallelize energy steering for training tokens at all positions through re-designed attention masks and positional indexes. **Left Panel:** the re-designed attention mask where 0 indicates blocking the cross-token-attention. **Right Panel:** the forward path with re-designed attention mask and positional index.

Energy Steering. EBFMs parallelize the energy steering at the sequence level for efficient large-scale pre-training. Specifically, for an input sequence $\boldsymbol{x}_{[1:S]} = [x_1, \ldots, x_S] \in \mathcal{X}$, EBFMs follow the causal attention paradigm such that latent state $\boldsymbol{z}_i$ can only attend to its prefix tokens $\boldsymbol{x}_{[1:i]}$. Therefore, the steering process for $\boldsymbol{z}_i$ is given by

$$\boldsymbol{z}_i^* = \arg\min_{\boldsymbol{z} \in \mathcal{R}^D} E(\boldsymbol{x}_{[1:i]}, \boldsymbol{z} \mid \theta). \tag{4}$$

However, separately steering each latent state $\boldsymbol{z}_1, \cdots, \boldsymbol{z}_S$ results in significant computational overhead during large-scale pre-training. To improve efficiency, EBFMs perform the steering for the latent states at all positions in parallel. To be concrete, EBFMs concatenate the training data $\boldsymbol{x}_{[1:S]}$ with latent states $\boldsymbol{z}_{[1:S]}$ into $[\boldsymbol{x}_{1:S}, \boldsymbol{z}_{1:S}]$, and simultaneously estimates energy values $E_1, \cdots, E_S$ corresponding to $[\boldsymbol{x}_{[1:1]}, \boldsymbol{z}_1], \cdots [\boldsymbol{x}_{[1:S]}, \boldsymbol{z}_S]$ within a single forward pass. The left panel of Figure 3 illustrates the attention mask for the concatenated sequence $[\boldsymbol{x}_{[1:5]}, \boldsymbol{z}_{[1:5]}]$, which restricts each latent state $\boldsymbol{z}_i$ to attend only to its prefix tokens $\boldsymbol{x}_{[1:i-1]}$ and itself. This design preserves the causal dependency required for language modeling while allowing all latent variables to be processed in parallel during training.

Beyond the attention masks, the positional indices of $\boldsymbol{z}_1, \boldsymbol{z}_2, \cdots, \boldsymbol{z}_S$ are assigned as $[1, 2, 3, \cdots, S]$ to align each latent state with its corresponding prediction position. These designs enable EBFMs to efficiently perform energy steering for all sequence positions in parallel, which follows a sequence-level optimization objective given by

$$\boldsymbol{z}_{[1:S]}^* = \arg\min_{\boldsymbol{z} \in \mathcal{R}^{S \times D}} \text{sum}_{1 \le i \le S} \Big(E_i(\boldsymbol{x}_{[1:S]}, \boldsymbol{z}_{[1:S]} \mid \theta) \Big). \tag{5}$$

EBFMs thereby approximate the solution through K-step MCMC sampling:

$$\boldsymbol{z}_{[1:S]}^k = \boldsymbol{z}_{[1:S]}^{k-1} - \alpha \nabla_{\boldsymbol{z}_{[1:S]}^{k-1}} \text{sum}_{1 \le i \le S} \Big(E_i(\boldsymbol{x}_{[1:S]}, \boldsymbol{z}_{[1:S]} \mid \theta) \Big), \tag{6}$$

where $\boldsymbol{z}_{[1:S]}^0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. The final state serves as an approximation to the optimal solution $\boldsymbol{z}_{[1:S]}^* \approx \boldsymbol{z}_{[1:S]}^K$.

Supervised Training. Given the input sequence $\boldsymbol{x}_{[1:S]}$ from the training corpora and optimized latent state $\boldsymbol{z}_{[1:S]}^*$ from the energy steering, EBFMs predict the subsequent token probabilities through the generative head $p(\boldsymbol{x}_{[2:S+1]}) = f_E(\boldsymbol{x}_{[1:S]}, \boldsymbol{z}_{[1:S]}^*; \boldsymbol{x}_{[2:S+1]} \mid \theta)$, and then update the model parameters θ to minimize the loss function:

$$\mathcal{L}(\theta) = -\log f_E(\boldsymbol{x}_{[1:S]}, \boldsymbol{z}_{[1:S]}^*; \boldsymbol{x}_{[2:S+1]} \mid \theta). \tag{7}$$

3.3 Algorithms

EBFM inference and training are given in Algorithms 1 and 2, respectively. Specifically, during inference, EBFMs perform energy steering (line 3) on the latent state, which in turn refines token generation (line 5). During training,

with each mini-batch of inputs, EBFMs perform energy steering (line 2), estimate loss value (line 3), and update model weights (line 4). Across both inference and training, energy steering is the key step that enables EBFMs to significantly capture in-context dependency for generating new tokens.

Algorithm 1 EBFM Inference

Input: input sequence $\boldsymbol{x}_{[1:S]}$.
Output: predicted next token $\hat{x}_{S+1}$.

1: $\boldsymbol{z}^0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** *layer* $k := 1$ to K **do**
3: Update latent state by Eq. (2) # Energy steering
4: **end for**
5: $\hat{x}_{S+1} \sim f_E(\boldsymbol{x}, \boldsymbol{z}^* \mid \theta)$ # Token generation
6: **return** $\hat{x}_{S+1}$

Algorithm 2 EBFM Pre-training

Input: pre-training dataset $\mathcal{D}$.
Output: EBFM model weights θ^*.

1: **for** mini-batches $\boldsymbol{x} \sim \mathcal{D}$ **do**
2: Update latent state by Eq. (6)
3: Estimate loss function $\mathcal{L}(\theta)$ given by Eq. (7)
4: Update θ by $\theta^* = \arg\min \mathcal{L}(\theta)$
5: **end for**
6: **return** θ^*

4 Evaluation Results

We pre-train EBFMs on large-scale corpora with billions of tokens. Evaluations of EBFMs comprehensively cover pre-training, supervised finetuning, zero-shot question-answering, zero-shot generations to assess their capabilities across diverse learning and reasoning tasks. We compare EBFMs with established foundation architectures, including GPTs and Diffusion Language Models (DLMs) [18].

4.1 Pre-training and Evaluation Recipe

Pre-training Recipe. We pre-train EBFMs on three large-scale corpora, including Fineweb [23], Climbmix [24], and DCLM [25]. To investigate scaling with model size, we vary the number of layers as $d \in \{12, 20, 26\}$ and jointly scale the model depth and hidden width following established Transformer scaling practices. Specifically, we follow the configuration of GPT-2 to have the hidden width $d_{\text{model}} = \gamma d$, where $\gamma = 64$, and number of attention heads $n_{\text{head}} = d_{\text{model}}/128$. Other hyper-parameter settings are provided in Table 1. After the pre-training, EBFMs are evaluated with three strategies as follows.

Scaling Law Evaluation. Scaling behaviors are evaluated on the validation sets of Fineweb, Climbmix, and DCLM datasets. Validation error is measured by bits per byte (BPB) defined by BPB $= (\log_2 \text{PPL})/(\text{Bytes per token})$, where PPL denotes the token-level perplexity on the validation datasets. BPB normalizes the PPL by the number of bytes, enabling more meaningful comparisons across different tokenizers. Lower BPB indicates better predictive performance. We evaluate the scaling behavior of models by observing how rapidly their validation error decreases with increasing training data and model size.

Zero-shot QA. Each foundation model is evaluated on 19 downstream tasks, including AGI-Eval LSAT-AR, ARC-Challenge, ARC-Easy, BIG-bench CS Algorithms, BIG-bench Dyck Languages, BIG-bench Language Identification, BIG-bench Operators, BIG-bench QA-Wikidata, BoolQ, CommonsenseQA, COPA, CoQA, HellaSwag, HellaSwag-ZeroShot, LAMBADA-OpenAI, OpenBookQA, PIQA, Winograd, and WinoGrande datasets. These tasks span diverse domains and capabilities, including science, algorithmic reasoning, language understanding, factual knowledge, logical reasoning, symbolic reasoning, and commonsense reasoning. Such broad evaluation provides a comprehensive assessment of the general-purpose capabilities of pretrained models.

Hyper-parameter	EBFM	Hyper-parameter	EBFM	Hyper-parameter	EBFM
Tokenizer	Nanochat	MCMC Steps	1	Weight Decay	0.2
Vocabulary Size	32,768	MCMC Step Size	300	Warmup Ratio	0.0057
Context Length	2,048	α Learning Rate	1.8	Warmdown Ratio	0.65
Global Batch Size	512	Muon Learning Rate	0.02	Muon Momentum	0.95
Optimizer	Muon + AdamW	Embedding Learning Rate	0.3	Muon β_2	0.95
Gradient Clipping	1	Final LR Fraction	0.05	AdamW β_1, β_2	0.8, 0.95

Table 1: Hyper-parameter setting for EBFM pre-training.

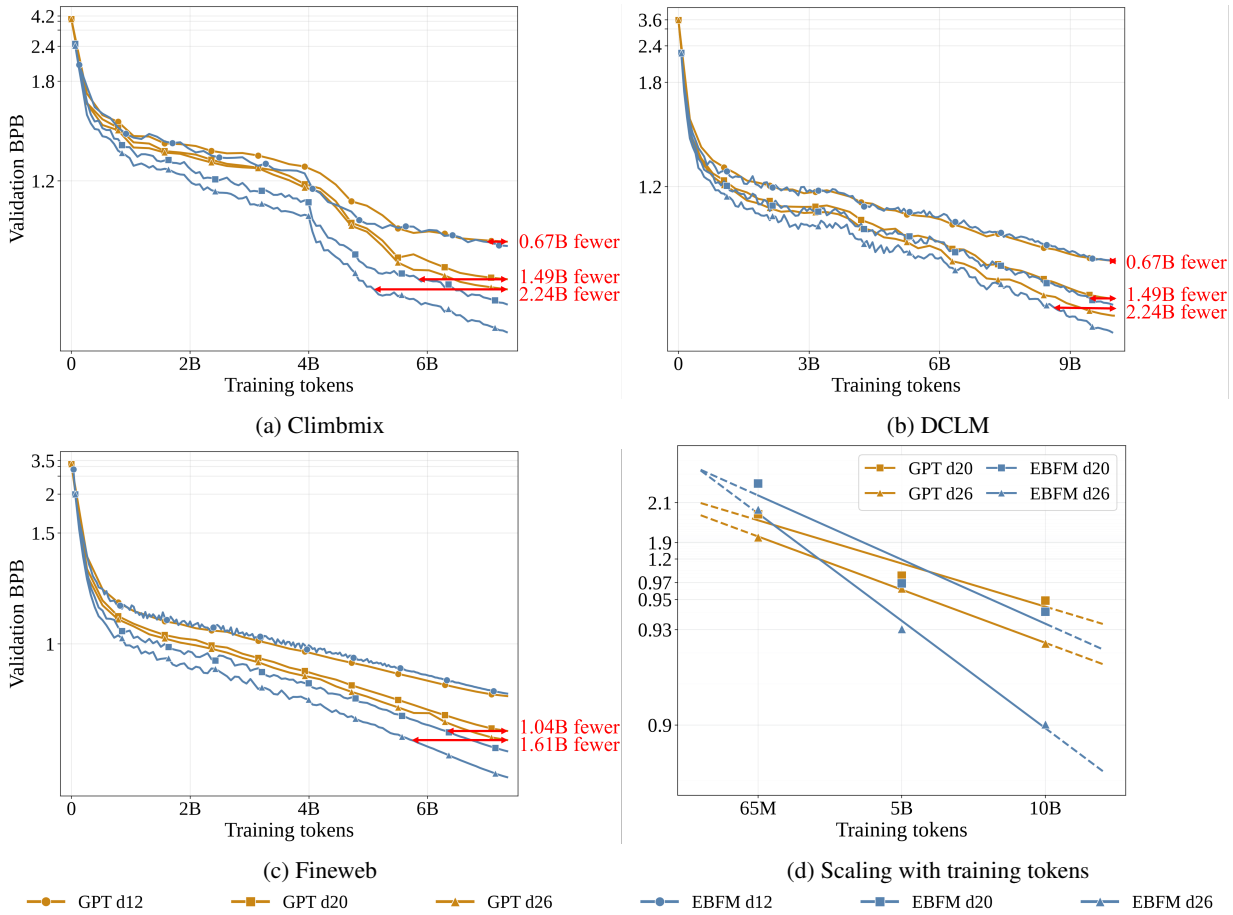


Figure 4: Validation error during pre-training on the (a) Climbmix, (b) DCLM, and (c) Fineweb datasets; (d) Scaling law of EBFMs versus different training budgets

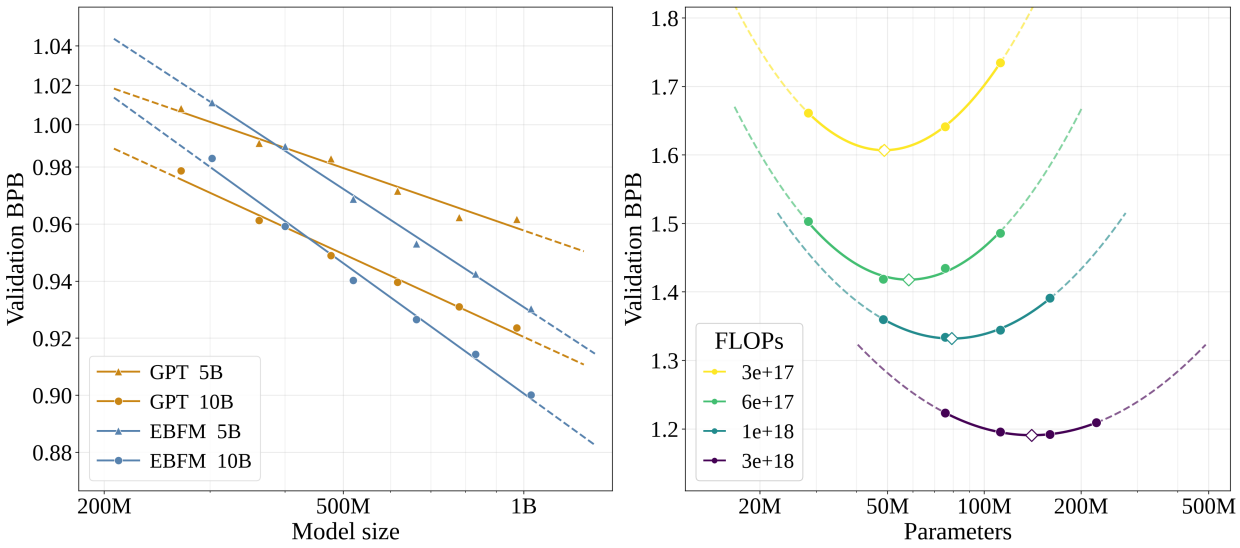
Zero-shot Generation. Each foundation model is pre-trained on the OpenWebText dataset [26] and evaluated on seven downstream datsests, including PTB [27], Wikitext [28], LM1B [29], Lambada [30], AG News [31], Pubmed, and Arxiv [32] datasets. Generation quality is measured by the validation BPB on each downstream dataset.

4.2 Evaluation of Scaling Law

To evaluate pre-training performance, we report the validation BPB versus training token number on the ClimbMix, DCLM, and FineWeb datasets in Figure 4 (a), (b), and (c), respectively. We analyze the performance in terms of training-token scaling, depth scaling, and IsoFLOP scaling as follows.

Data Scaling. Across all three corpora, EBFMs consistently outperform GPTs, with BPB decreasing more rapidly as the number of training tokens increases. Given the same training tokens, EBFMs consistently achieves lower BPB than GPTs across different model sizes. In details, as shown in Figure 4 (a), on the ClimbMix dataset, EBFM with depths of 12, 20, and 26 layers can match GPTs trained on 7B tokens after pre-training on 4.5B, 3.5B, and 5.1B tokens, respectively. Thus, EBFMs require 0.67B, 1.49B, and 2.24B fewer training tokens, respectively. Similarly, on the DCLM and Fineweb datasets, EBFMs significantly 1.04B and 1.61B fewer tokens than GPTs, respectively, to match the final performance of GPTs. These results demonstrate the potential of ***EBFM as a data-efficient foundation architecture.***

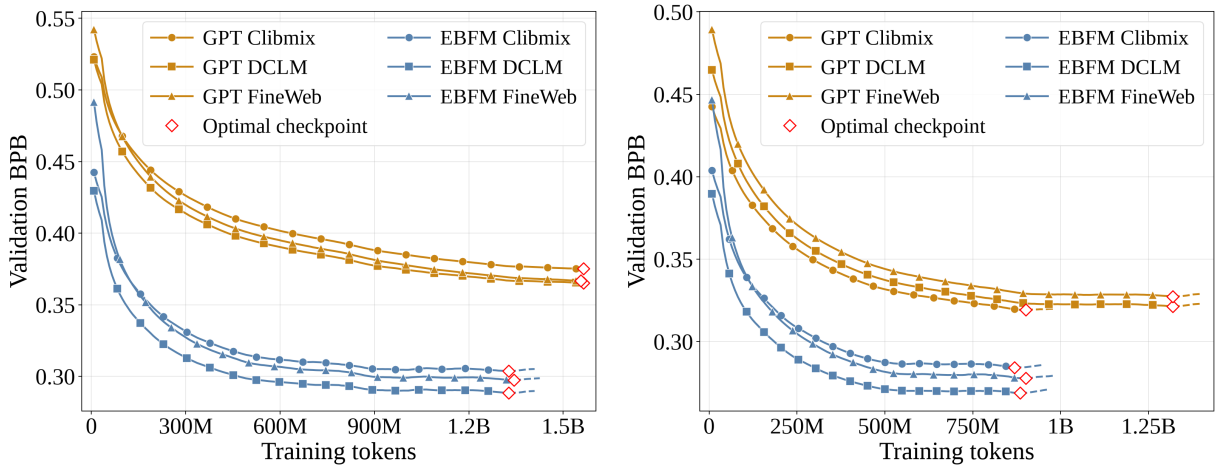
Moreover, Figure 4 (d) illustrates the minimal validation error versus training tokens on the DCLM dataset with different training budgets. As the training tokens increase from 65M to 10B, the validation error of EBFMs decreases faster than that of GPTs. Thus, ***EBFMs exhibit an increasing performance advantage over GPTs as the training budget grows.*** These results establish EBFMs as a promising architecture for large-scale pre-training.



(a) Minimal validation BPB versus model depth

(b) IsoFLOPs curves

Figure 5: Scaling behaviors of EBFM with respect to model depth and training FLOPs.



(a) $d20$

(b) $d26$

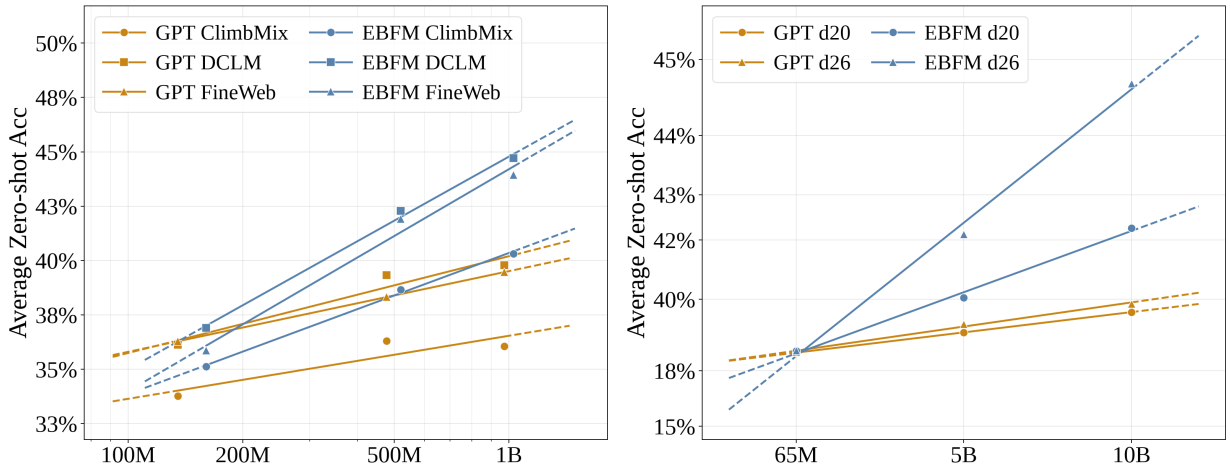
Figure 6: Validation error during supervised fine-tuning. (a) $d20$ models (b) $d26$ models

Parameter Scaling. Figure 5 (a) shows the validation BPB versus model size, which are measured on the DCLM dataset with 5B and 10B training tokens. As the model size increases from 268M to 1.03B parameters, the validation error of EBFMs reduces faster than that of GPTs. ***EBFM's performance advantage over GPTs becomes increasingly pronounced at larger model scales.*** This result demonstrates that EBFMs benefit more effectively from parameter scaling, highlighting their potential as a foundation architecture.

IsoFLOP Law. Figure 5 (b) presents the U-shape IsoFLOP curves of EBFMs on the DCLM dataset, where each curve represents the validation BPB across different model sizes under fixed training FLOPs. The minimal point of each curve identifies the compute-optimal model size for EBFMs. As the training budget increases, the optimal point shifts downward and to the right, indicating that larger EBFMs achieve lower validation error. This trend demonstrates promising scaling behaviors of EBFMs like GPTs [33]. To be concrete, it suggests that ***EBFM can consistently achieve lower validation error with increased training budget when model size and training tokens are scaled appropriately.*** The IsoFLOP curves highlight EBFM's potential as a foundation architecture.

Pretrain data	Model	Depth	AGIEval	ARC		BIG-bench					BoolQ	CSQA	COPA	CoQA	HellaSwag		LAMBADA	OBQA	PIQA	Winograd		Avg.
			LSAT-AR	C	E	CSAlg	Dyck	LangID	Ops	WikiQA					10-shot	0-shot				WSC	WinoG	
ClimbMix	GPT	$d12$	**26.96**	23.20	42.80	40.80	11.20	**25.80**	11.90	28.20	55.60	21.80	56.00	15.00	**31.00**	30.20	**31.20**	26.40	57.00	**56.41**	**50.20**	33.77
		$d20$	23.91	**26.60**	47.60	41.00	13.20	**26.20**	15.71	34.20	**57.60**	**28.00**	**62.00**	19.80	34.00	34.00	35.80	25.40	59.60	59.34	46.00	36.31
		$d26$	26.52	27.80	46.60	37.40	**14.20**	**26.00**	17.14	37.00	50.40	**25.00**	59.00	20.80	33.40	32.60	35.00	27.20	60.00	57.51	51.60	36.06
	EBFM	$d12$	21.74	**23.29**	**44.36**	**42.12**	**13.70**	25.63	**18.57**	**30.75**	**56.24**	**37.84**	**58.00**	**16.47**	30.47	**30.75**	28.18	**27.20**	**58.16**	54.58	49.33	**35.13**
		$d20$	**24.35**	26.02	**50.80**	**41.59**	**17.50**	24.83	**21.90**	**42.55**	56.61	25.72	61.00	**26.92**	**36.70**	**36.60**	**38.93**	**27.00**	**61.21**	**64.47**	**50.12**	**38.67**
		$d26$	**30.00**	**29.61**	**53.87**	**40.53**	11.90	25.00	**19.52**	**47.66**	**52.97**	23.59	**65.00**	**26.31**	**40.16**	**40.23**	**43.12**	**30.20**	**63.76**	**66.30**	**56.27**	**40.32**
DCLM	GPT	$d12$	24.78	25.00	47.60	37.40	12.60	23.80	**13.33**	30.20	**55.20**	30.40	**63.00**	**14.00**	32.00	31.60	**38.40**	28.40	64.40	**61.54**	**52.80**	36.13
		$d20$	**26.96**	28.00	53.20	**44.40**	12.00	**25.80**	13.81	41.80	55.00	23.00	**66.00**	19.20	38.00	39.00	44.80	30.80	68.00	64.84	52.60	39.33
		$d26$	23.04	28.40	54.20	38.60	18.00	23.60	17.14	44.00	52.40	**26.00**	61.00	17.60	42.00	42.20	48.60	31.00	70.40	63.74	54.00	39.79
	EBFM	$d12$	**26.09**	**25.60**	**48.36**	**44.09**	**21.60**	**25.58**	12.86	**33.77**	50.92	**34.81**	57.00	11.62	**34.27**	**34.08**	33.30	**29.60**	**65.45**	59.71	52.41	**36.90**
		$d20$	22.61	**29.78**	**57.74**	41.89	**16.30**	24.79	**17.62**	**48.90**	**55.02**	**32.35**	61.00	**22.44**	**47.92**	**47.21**	**48.09**	**33.00**	**69.91**	**73.63**	**53.28**	**42.29**
		$d26$	**26.96**	**32.25**	**61.78**	**44.02**	**18.30**	**24.40**	**18.10**	**54.25**	**55.29**	25.88	**65.00**	**26.53**	**54.70**	**53.64**	**52.94**	**35.20**	**72.42**	**71.43**	**56.35**	**44.71**
FineWeb	GPT	$d12$	21.30	24.60	53.20	**40.80**	9.00	**25.60**	11.90	**34.40**	**56.80**	30.80	**62.00**	**15.20**	**35.00**	**34.80**	**31.60**	30.80	62.80	53.48	**55.40**	**36.29**
		$d20$	22.61	32.60	57.40	41.20	10.40	23.20	13.33	40.00	**50.40**	31.00	62.00	18.80	40.40	39.40	33.20	34.00	66.20	60.07	51.80	38.32
		$d26$	24.35	30.60	60.60	41.00	8.40	**28.80**	15.24	45.40	**58.60**	23.80	63.00	16.80	40.20	40.80	36.80	35.00	67.60	61.90	51.00	39.47
	EBFM	$d12$	**24.78**	**27.47**	**54.00**	34.85	**11.80**	24.93	**12.38**	32.79	48.53	**32.68**	61.00	12.48	34.68	34.31	28.82	**32.20**	**64.96**	**58.24**	50.36	35.86
		$d20$	**29.13**	**35.24**	**63.22**	**44.62**	**13.20**	**24.82**	**16.67**	**48.05**	47.37	**35.54**	**67.00**	**21.01**	**45.39**	**45.38**	**36.46**	**35.40**	**68.06**	**66.30**	**53.59**	**41.92**
		$d26$	**26.52**	**36.95**	**66.96**	**43.86**	**16.00**	24.73	**16.67**	**50.52**	57.83	**30.06**	**68.00**	**24.59**	**51.21**	**50.93**	**40.64**	**38.40**	**70.02**	**65.93**	**54.85**	**43.93**

Table 2: Zero-shot accuracy (%) on 19 question-answering datasets after pre-training.



(a) Model Size

(b) Training Tokens

Figure 7: (a) Zero-shot accuracy versus model size (b) Zero-shot accuracy versus training tokens on DCLM dataset

4.3 Evaluation of Supervised Fine-tuning

We further evaluate the performance of EBFMs after supervised fine-tuning (SFT). The SFT takes the Nanochat-SFT dataset, which is a mixture of SmolTalk, MMLU, and GSM8K datasets [34]. For both GPTs and EBFMs, SFT is initialized from their respective optimal pre-training checkpoints, ensuring that each architecture starts from its optimal pre-trained state. Figure 6 illustrates the validation error versus the training token numbers for $d20$ and $d26$ models. During SFT, EBFMs achieve lower validation error while consuming fewer SFT tokens, indicating that their favorable pre-trained initialization enables more data-efficient fine-tuning. This highlights the advantage of EBFMs over GPTs throughout the full training pipeline, from pre-training to SFT, establishing its potential as a foundation architecture.

4.4 Evaluation of Question-Answering Tasks

We evaluate EBFMs on zero-shot question-answering (QA). The details of the 19 downstream QA tasks are given in Section 4.1. According to the results in Table 2, EBFMs significantly outperform GPTs on most downstream tasks. These results highlight the potential of EBFM to serve as foundation models for general-purpose tasks across diverse application scenarios. Moreover, Figure 7 (a) and (b) illustrate the zero-shot accuracy versus model size and training tokens, respectively. The zero-shot performance of EBFMs improves more rapidly with increasing model size and training tokens than that of GPTs; and the advantage of EBFMs grows at larger scales. These results demonstrate the favorable scaling behaviors of EBFM and its stronger potential at larger scales.

Model	OpenWebText-val	PTB	Wikitext	LM1B	Lambada	AG News	Pubmed	Arxiv	Average
EBFM	**0.93**	**1.46**	**1.10**	**1.24**	1.53	**1.34**	**1.34**	**1.44**	**1.35**
GPT	0.94	1.51	1.11	1.26	**1.48**	1.35	1.39	1.51	1.37
MDLM	1.17	1.53	1.23	1.38	**1.48**	1.52	1.41	1.50	1.44

Table 3: Validation BPB of EBFMs, GPTs, and MDLM pre-trained on the OpenWebText datasests, where EBFMs and GPTs are trained for only ONE epoch with 9.06B tokens, and MDLM is trained for TWO epoches with 18.12B tokens. Average values are estimated for zero-shot settings.

4.5 Zero-shot Generation

EBFMs are compared with GPTs, and Diffusion Large Models (MDLMs) for zero-shot generations. All models are pre-trained on the OpenWebText dataset [26], where EBFMs and GPTs are trained for ONE epoch with 9.06B tokens, and MDLMs are trained for TWO epochs with 18.12B tokens. All models are evaluated on the OpenWebText validation dataset as well as seven downstream datasets, including PTB, Wikitext, LM1B, Lambada, AG News, Pubmed, and Arxiv datasets. The evaluation results are given in Table 3, where EBFMs consistently outperform GPTs and MDLMs, achieving 0.02 and 0.09 lower BPB errors, respectively. The evaluation results of zero-shot generation demonstrate the potential of EBFMs as a foundation architecture.

5 Open-source Release

We fully release the training infrastructure and training logs at https://github.com/datamllab/openesm, the model weights at https://huggingface.co/collections/guan-wang/openesm. All datasets in this work are publicly available and properly linked in our released code. We sincerely hope the community can reproduce our results and jointly explore the potential of energy-based foundation models.

6 Limitations under Current Ecosystem

Efficiency. Our largest EBFM configuration is $d26$, with 1.03B parameters. Pre-training efficiency remains a major bottleneck to scaling EBFM to larger sizes. Currently, the training of EBFM and other EBMs relies primarily on distributed data parallelism (DDP), where each GPU must maintain a complete model backbone. Existing model parallel infrastructures, including FSDP, Megatron-LM, and DeepSpeed ZeRO, do not yet readily support the efficient second-order differentiation required by EBM training. Therefore, a single model cannot be easily sharded across multiple GPUs, which limits the model size to the memory capacity of a single H200 GPU. We encourage the community to extend distributed-training infrastructure to support second-order gradients and model sharding. Addressing this bottleneck will enable EBMs to scale to larger capacities and better performance.

Block-wise Generation. Block-wise generation enables safer and more controllable generation by jointly optimizing whole sentence under energy-based guidance. Different from token-wise generation, which generates a locally optimal token at each step, block-wise energy evaluation enables semantic coherence and safety constraints to be assessed over a longer span. Therefore, block-wise energy steering facilitates improving long-range contextual consistency and constraint compliance. Extending EBFMs to block-wise generation remains an important direction for future work.

7 Conclusion

In this work, we propose EBFMs as a foundation architecture, and explore its scaling behaviors across general-purpose tasks. Specifically, EBFMs perform energy steering to optimize hidden states during inference, thereby guiding token generation toward outputs with lower energy. Moreover, we develop parallel energy steering for efficient EBFM pre-training, and further pre-train the models on three large-scale corpora. Across multiple evaluation settings and 19 downstream tasks spanning diverse domains, EBFMs demonstrate promising scaling performance with increased model size and training cost. These findings highlight the substantial potential of EBFMs as a foundation architecture for general-purpose tasks. We completely release our infrastructure and model weights to enable full reproduction of our results. We encourage the community to jointly explore the potential of energy-based foundation models.

8 Future Directions

We believe that the potential of EBFMs remains under-explored, and we welcome the community to investigate promising directions, including but not limited to the following:

- **Supervised Fine-tuning.** Fine-tuning EBFMs on domain-specific instructions and data to acquire to strengthen their capabilities in specialized domains, such as scientific reasoning, code generation, and finance. Effective fine-tuning of EBFMs could further improve their adaptability to diverse applications.
- **Efficient Inference/Training.** EBFMs differ architecturally from GPTs, making existing efficient inference or training, such as GPTQ, Flash Attention, and Page Attention, difficult to apply. Developing efficient inference and training algorithms tailored to the architecture of EBFM will be promising directions.
- **Training Infrastructure.** Developing distributed training infrastructure to scale up EBFM training. In particular, future systems could support model sharding across multiple GPUs, memory-efficient computation of higher-order gradients that are comparable with frameworks such as FSDP and Megatron-LM.

References

[1] Yann LeCun, Sumit Chopra, Raia Hadsell, M Ranzato, Fujie Huang, et al. A tutorial on energy-based learning. *Predicting structured data*, 1(0), 2006.

[2] Patrick Huembeli, Juan Miguel Arrazola, Nathan Killoran, Masoud Mohseni, and Peter Wittek. The physics of energy-based models. *Quantum Machine Intelligence*, 4(1):1, 2022.

[3] Yilun Du and Igor Mordatch. Implicit generation and modeling with energy based models. *Advances in neural information processing systems*, 32, 2019.

[4] Davide Carbone. Hitchhiker's guide on the relation of energy-based models with other generative models, sampling and statistical physics: a comprehensive review. *arXiv preprint arXiv:2406.13661*, 2024.

[5] Yang Song and Diederik P Kingma. How to train your energy-based models. *arXiv preprint arXiv:2101.03288*, 2021.

[6] Igor Mordatch. Concept learning with energy-based models. *arXiv preprint arXiv:1811.02486*, 2018.

[7] Yilun Du, Shuang Li, Joshua Tenenbaum, and Igor Mordatch. Learning iterative reasoning through energy minimization. In *International Conference on Machine Learning*, pages 5570–5582. PMLR, 2022.

[8] Jianwen Xie, Song-Chun Zhu, and Ying Nian Wu. Learning energy-based spatial-temporal generative convnets for dynamic patterns. *IEEE transactions on pattern analysis and machine intelligence*, 43(2):516–531, 2019.

[9] Ou Zhijian. Energy-based models with applications to speech and language processing. *Foundations and Trends® in Signal Processing*, 18(1-2):1–199, 2024.

[10] Alexi Gladstone, Ganesh Nanduru, Md Mofijul Islam, Aman Chadha, Jundong Li, and Tariq Iqbal. Cognitively inspired energy-based world models. *arXiv preprint arXiv:2406.08862*, 2024.

[11] Rinu Boney, Juho Kannala, and Alexander Ilin. Regularizing model-based planning with energy-based models. In *Conference on Robot Learning*, pages 182–191. PMLR, 2020.

[12] Federico Benzi, Federica Ferraguti, Giuseppe Riggio, and Cristian Secchi. An energy-based control architecture for shared autonomy. *IEEE Transactions on Robotics*, 38(6):3917–3935, 2022.

[13] Nikolaos Gkanatsios, Ayush Jain, Zhou Xian, Yunchu Zhang, Christopher Atkeson, and Katerina Fragkiadaki. Energy-based models are zero-shot planners for compositional scene rearrangement. *arXiv preprint arXiv:2304.14391*, 2023.

[14] David Duvenaud, Jackson Wang, Jorn Jacobsen, Kevin Swersky, Mohammad Norouzi, and Will Grathwohl. Your classifier is secretly an energy based model and you should treat it like one. *ICLR 2020*, 2020.

[15] Rui Liu, Wenguan Wang, and Yi Yang. Vision-language navigation with energy-based policy. *Advances in Neural Information Processing Systems*, 37:108208–108230, 2024.

[16] Cheng Qi, Zhiyong Feng, Meng Xing, Yong Su, Jinqing Zheng, and Yiming Zhang. Energy-based temporal summarized attentive network for zero-shot action recognition. *IEEE Transactions on Multimedia*, 25:1940–1953, 2023.

[17] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

[18] Subham S Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. *Advances in Neural Information Processing Systems*, 37:130136–130184, 2024.

[19] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[20] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *Advances in Neural Information Processing Systems*, 38:50608–50646, 2026.

[21] Sam Bond-Taylor, Adam Leach, Yang Long, and Chris G Willcocks. Deep generative modelling: A comparative review of vaes, gans, normalizing flows, energy-based and autoregressive models. *IEEE transactions on pattern analysis and machine intelligence*, 44(11):7327–7347, 2021.

[22] Alexi Gladstone, Ganesh Nanduru, Md Mofijul Islam, Peixuan Han, Hyeonjeong Ha, Aman Chadha, Yilun Du, Heng Ji, Jundong Li, and Tariq Iqbal. Energy-based transformers are scalable learners and thinkers. *arXiv preprint arXiv:2507.02092*, 2025.

[23] Guilherme Penedo, Hynek Kydlíček, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, Thomas Wolf, et al. The fineweb datasets: Decanting the web for the finest text data at scale. *Advances in Neural Information Processing Systems*, 37:30811–30849, 2024.

[24] Shizhe Diao, Yu Yang, Yonggan Fu, Xin Dong, Dan Su, Markus Kliegl, Zijia Chen, Peter Belcak, Yoshi Suhara, Hongxu Yin, et al. Nemotron-climb: Clustering-based iterative data mixture bootstrapping for language model pre-training. *Advances in Neural Information Processing Systems*, 38, 2026.

[25] Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Gadre, Hritik Bansal, Etash Guha, Sedrick Keh, Kushal Arora, et al. Datacomp-lm: In search of the next generation of training sets for language models. *Advances in Neural Information Processing Systems*, 37:14200–14282, 2024.

[26] Aaron Gokaslan and Vanya Cohen. Openwebtext corpus. http://Skylion007.github.io/OpenWebTextCorpus, 2019.

[27] Mitch Marcus, Beatrice Santorini, and Mary Ann Marcinkiewicz. Building a large annotated corpus of english: The penn treebank. *Computational linguistics*, 19(2):313–330, 1993.

[28] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016.

[29] Ciprian Chelba, Tomas Mikolov, Mike Schuster, Qi Ge, Thorsten Brants, Phillipp Koehn, and Tony Robinson. One billion word benchmark for measuring progress in statistical language modeling. *arXiv preprint arXiv:1312.3005*, 2013.

[30] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc-Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The lambada dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: Long papers)*, pages 1525–1534, 2016.

[31] Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. *Advances in neural information processing systems*, 28, 2015.

[32] Arman Cohan, Franck Dernoncourt, Doo Soon Kim, Trung Bui, Seokhwan Kim, Walter Chang, and Nazli Goharian. A discourse-aware attention model for abstractive summarization of long documents. In *Proceedings of the 2018 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 2 (short papers)*, pages 615–621, 2018.

[33] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

[34] Andrej Karpathy. nanochat: The best chatgpt that $100 can buy, 2025.